OBJECT DESTRUCTION

- Most object-oriented languages have a method called `***destructor***`, which is called to destroy an object.
- Destroying an object means freeing the resources that have already been allocated to it by the ***constructor***.
- An object is destroyed when it no longer references any memory location (object = null).
- In Java, object destruction is handled by an automatic mechanism called the garbage collector (GC).
- Memory reclamation can be explicitly invoked by the programmer at specific times using the `***System.gc()***` command.

© Soumaya Trabelsi

EXAMPLE

```
class Date {

        private int d; private int m; private int y;


        Date() {d=1;m=1;y=1900;}
        Date(int day,int month,int year) {d=day; m=month; y=year;}
        void Show( ) { System.out.println(d + "/" + m + "/" + y) ; }
        public static void main(String[ ] args)
        {
                Date dt = new Date(1,6,1997);
                Date dt1= new Date( );
                dt.Show( );
                dt =null; // "dt" no longer designates anything; it no longer has a
reference. It is no longer accessible. At any moment, the GC can destroy the object.
                System.gc( ); // Implicit call from the GC

        }
}
```

@sour

EXERCISE: TIME

Create a **Time** class containing three integer attributes: ***H***, ***M***, and ***S***, and the following methods:

1. The following constructors:
   - ***Time(int hour)***: constructs a Time object from a given hour.
   - ***Time(int hour, int minute)***: constructs a Time object from a given hour and minute.
   - ***Time(int hour, int minute, int second)***: constructs a Time object from the hour, minute, and second values.
2. A ***display()*** method that displays the time in the following format: "It is h hours m minutes s seconds".
3. A ***main*** method that creates a time and displays it.

THE "CURRENT" OBJECT

- The keyword `***this***` represents a reference to the current object, that is, the object containing the method that is currently being executed.
- The `***this***` reference can be useful:
  - when a local variable (or parameter) "hides," by having the same name, an attribute of the class.
  - when a reference to the current object is not yet known.

SELF-REFERENCE: THIS

The keyword ***this*** stands for:

- clear up ambiguities
- reuse a constructor (call another constructor of the same class. This is written in the first statement of the constructor)

```
public class Point {
// …
public Point (double x, double y)
        {
                x = x;
                y = y;
        }
public Point (){
                x=0;
                y=0;
        }
}
```

ambiguity!

redundancy

```
public class Point {
// …
public Point (double x, double y)
        {
                this.x = x;
                this.y = y;
        }
public Point (){
                this(0, 0);
        }
}
```

© Soumaya Trabelsi